Recreating Apollo Guidance Computer in ARM Cortex-M0

M. Ahsan Al Mahir

Department of Computer Science
University of Oxford

Supervisor: Prof. Alex Rogers

A project report submitted for the degree of Master of Mathematics and Computer Science

Trinity 2025

986 words (using `texcount`)

Abstract

This is the abstract.

Contents

§1 Introduction

(i) Project overview and objectives

(ii) Significance of the Apollo Guidance Computer in computing history

(iii) Your motivation for basing a modern OS on the AGC architecture

(iv) Brief summary of your implementation and innovations

§2 The Apollo Guidance Computer: Historical Context

(i) Brief history of the AGC development

(ii) Technical specifications and constraints

(iii) Key design principles and innovations

(iv) Notable features of the original operating system

The Apollo Guidance Computer (AGC) was a pioneering digital computer developed in the early 1960s to navigate and control the Apollo spacecraft during their missions to the Moon. Installed aboard both the Command Module and the Lunar Module, the AGC was integral to the success of the Apollo program, providing real-time guidance, navigation, and control capabilities.

Designed by the MIT Instrumentation Laboratory under the leadership of Charles Stark Draper, the AGC featured a 16-bit word length, with 15 data bits and one parity bit, and operated at a frequency of 2.048 MHz. The computer's memory was divided into 2,048 words of erasable magnetic-core memory (RAM) and 36,864 words of fixed core rope memory (ROM), which stored the software and mission data.

Astronauts interacted with the AGC through the Display and Keyboard (DSKY) interface, a 21-digit display and 19-button keyboard that allowed them to input commands and receive information. This interface was crucial for manual adjustments and monitoring during the missions.

The AGC's software was written in AGC assembly language and stored on rope memory. The bulk of the software was on read-only rope memory and thus could not be changed in operation, but some key parts of the software were stored in standard read-write magnetic-core memory and could be overwritten by the astronauts using the DSKY interface, as was done on Apollo 14.

2.1 AGC Operating System Architecture

(i) Core components and structure

(ii) Memory management approach

(iii) Executive system and task scheduler

(iv) Interrupt handling system

(v) I/O subsystem

2.2 Operating System

The Apollo Guidance Computer (AGC) utilized a specialized operating system to manage its resources and execute the software required for the Apollo missions. The operating system, known as AGC software, was designed to perform real-time control and navigation tasks with

extreme reliability and efficiency. Unlike modern operating systems, which handle general-purpose tasks for a wide variety of applications, the AGC's operating system was dedicated exclusively to the guidance, navigation, and control of the Apollo spacecraft.

One of the key innovations of the AGC operating system was its use of a *"priority scheduling"* system. This system ensured that higher-priority tasks, such as navigation corrections or abort sequences, were processed before lower-priority tasks, allowing the AGC to respond dynamically to changing conditions during the mission. The operating system also utilized memory efficiently, with the available 2,048 words of RAM and 36,864 words of ROM being carefully allocated to critical software routines.

The AGC operating system also incorporated error-checking mechanisms to maintain reliability in the face of potential hardware malfunctions. Given the harsh and unpredictable environment of space, the system was designed to operate autonomously, with minimal input from the astronauts, while still allowing for manual intervention when necessary. The operating system's software was written in AGC assembly language and was stored in the AGC's rope memory, providing a highly optimized and space-efficient solution for spacecraft operations.

This operating system consisted of two parts: the ***Executive***, a batch job-scheduler that used priority based cooperative multi-tasking to manage longer running jobs, and the ***Waitlist***, a timer interrupt-driven pre-emptive scheduler that managed real-time tasks running on tight timing constraints. By design, waitlist tasks were run in an interrupt-inhibited mode, ensuring uninterrupted processing. This greatly limited how much computation these tasks are able to do, limiting each task to about 150-200 instructions. As a result, a waitlist task requiring extensive calculation would have to schedule other waitlist tasks, or a longer running executive job. On the other hand, to meet the deadlines on critical tasks, executive jobs must schedule waitlist tasks to be run within a certain deadline.

The operating system also consisted of a robust recovery mechanism, that offered different levels of restart protection for various tasks and jobs.

The success of the AGC operating system relied on the cooperation between the executive jobs and waitlist tasks and it's robust recovery system.

2.3 The Executive

The ***Executive*** in the Apollo Guidance Computer is a multiprocessing preemptive real-time operating system. It has a few features that set it apart from modern RTOS's, and made it such a visionary piece of technology of its time.

In this section, we'll give a brief overview of the features of the Executive.

- Cooperative and preemptive
- Process construction:
- What we'd call process descriptors, were called Core Set.
- Could ask for dyanmic storage at the point of process creation.

- Priority based scheduing
- Context switching: using the `NEWJOB` register
- Process sleep and wakeup
- Waitlist to do most of the real time operations
- Restart and recovery

§3 micro:bit v1 Architecture and Specifications

(i) Hardware overview (nRF51822 microcontroller, memory, I/O)

(ii) Constraints and capabilities

(iii) Comparison with AGC hardware

  (i) Processing power

  (ii) Memory availability

  (iii) I/O capabilities

  (iv) Power requirements

(iv) Development environment and toolchain

§4 Implementation Architecture and Design

(i) Overall system architecture

(ii) Design principles and approach

(iii) Adaptations for micro:bit constraints

(iv) High-level organization and module relationships

§5 Implementation Details

5.1 Core Executive System

(i) Task scheduling and prioritization
(ii) Interrupt handling
(iii) Time management
(iv) Original AGC features preserved vs. modified

5.2 Memory Management

(i) Memory organization and allocation
(ii) FRAM extension implementation
(iii) Comparison with AGC's memory management

5.3 I/O Subsystem

(i) Interface with micro:bit peripherals
(ii) Mapping AGC I/O concepts to micro:bit
(iii) Novel I/O approaches

5.4 Key Algorithms and Procedures

(i) Implementation of critical AGC algorithms
(ii) Performance optimizations
(iii) Code examples of significant components

§6 Evaluation and Testing

(i) Testing methodology on micro:bit
(ii) Performance metrics
(iii) Reliability considerations
(iv) Comparison with original AGC capabilities
(v) Limitations and trade-offs

§7 Future Work

(i) Potential improvements and extensions

(ii) Unresolved challenges

(iii) Research or application possibilities

§8 Conclusion

(i) Summary of achievements

(ii) Reflections on the AGC's enduring design principles

(iii) Broader implications for OS design

Reference

§A Appendices

(i) Code snippets of key algorithms

(ii) Detailed specifications

(iii) Test results